Gorch on Microcontrollers

Test Harness, Real ESP32-C3 Emulation, and Trained Iris Classifier

Branch `claude/gorch-microcontroller-inference-Uw9vI`

2026-04-18

Summary

A single int8 inference kernel from gorch (`embedded/gm1_avr.h`) now runs across **three independent embedded targets** and is validated by Go test harnesses that boot a simulated MCU, parse UART output, and assert it matches a Go-side reference byte-for-byte. The newest target uses **Espressif's QEMU fork** — real ESP32-C3 emulation, not a generic RISC-V board. The model deployed on RISC-V and ESP32-C3 is a **trained Iris classifier** that achieves 98.9% accuracy on the full Iris dataset when running through the simulated firmware.

Target	Emulator	Toolchain	Model	Test time
ATmega2560 (AVR-8)	simavr 1.6	gcc-avr 7.3	4-6-3 fixture	~1.2 s
RISC-V virt (rv32)	qemu-system-riscv32	riscv64-unknown-elf-gcc 13	Iris 4-8-3	~1.0 s
ESP32-C3 (real SoC)	**Espressif QEMU 9.2.2**	**riscv64-unknown-elf-gcc 13**	**Iris 4-8-3**	**~0.6 s**

What is being tested

For each target the harness performs the same end-to-end loop:

1. **Pick or train a model.** AVR uses a fixture with hand-picked weights; RISC-V and ESP32-C3 train an Iris MLP from scratch in pure Go on every run.
2. **Quantize to int8.** Post-training, symmetric, per-tensor quantization. Activations are calibrated by running the training set through the float model. The resulting Q0.31 multiplier `M` and shift `S` per layer match the kernel's `gm1_requant` byte-for-byte.
3. **Emit C headers.** `iris_model.h` (weights, biases, `M`, `S`, dimensions) and `iris_input.h` (the test sample as int8) are regenerated by the test before each build.
4. **Build firmware.** A minimal bare-metal C/asm program (~100 LOC) that links the kernel against the freshly emitted headers. No SDK, no FreeRTOS — just startup, UART driver, and `main`.
5. **Boot in the emulator.** Go calls the appropriate QEMU/simavr binary, captures UART output, kills the process as soon as the `CLASS ...` line arrives.

6. **Compare.** The test asserts firmware class + every logit equals the Go-side `TinyModel.Infer(...)` exactly. For the full-dataset variant (RISC-V) the assertion also checks the trained model's accuracy on real Iris samples against a 0.90 threshold.

The shared kernel (`embedded/gm1_avr.h`)

A header-only C file, ~100 lines, with three ops:

- `gm1_linear_i8`: fully-connected layer with int8 weights and activations, int32 accumulator, fused requant back to int8.
- `gm1_relu_i8`: in-place ReLU.
- `gm1_linear_i32`: classifier tail — same as `linear_i8` but leaves the int32 accumulator alone so argmax runs on full precision.
- `gm1_argmax_i32`: returns the index of the largest logit.

Two design choices make it portable across all three targets:

- **Flash-vs-SRAM abstraction.** `GM1_RD_I8(p)` and `GM1_RD_I32(p)` expand to `pgm_read_byte(p)` / `pgm_read_dword(p)` on AVR (where weights must be fetched from flash with a separate instruction) and to plain dereferences on RISC-V (where flash is memory-mapped). The firmware files for all three targets `#include "../../gm1_avr.h"` unchanged.
- **No floats.** AVR has no FPU; ESP32-C3's RISC-V `M` extension gives us 32-bit multiply but no float; we use the same int32-accumulator, Q0.31-multiplier requantize on every target.

The Iris classifier

The "useful model" deployed on the RISC-V and ESP32-C3 targets is a small MLP trained on Fisher's Iris dataset:

- **Architecture**: 4 features → 8 hidden (ReLU) → 3 classes.
- **Dataset**: 90 samples, balanced (30 per class), embedded in `embedded/iris/data.go`. Public domain.
- **Training**: pure Go SGD with He init and softmax cross-entropy loss. 300 epochs over 72 training samples (80/20 stratified split). Deterministic — same seed produces the same weights.
- **Quantization**: post-training, symmetric per-tensor. Weight scale = `max_abs(W) / 127`; bias scaled into the int32 accumulator's units; activation scale calibrated from training-set forward passes.

Float training reaches 100% on the training set and **94.4% on the held-out test set** (17/18). The int8 quantized model **agrees with the float model on 100% of samples** (90/90 dataset-wide). When the deployment loop is closed by running every Iris sample through the ESP32-class firmware in QEMU, **firmware accuracy is 89/90 = 98.9%**.

Real ESP32-C3 emulation

Earlier in the session the harness used upstream QEMU's generic `-machine virt` board, which is RISC-V but not specifically ESP32-C3 (different memory map, different UART, different boot

story). The new ESP32-C3 harness uses **Espressif's QEMU fork**:

- Binary downloaded from `github.com/espressif/qemu/releases/esp-develop-9.2.2-20250817` (16 MB tarball; not in any distro repo).
- `-machine esp32c3` — real ESP32-C3 SoC model: rv32imc CPU, IRAM at `0x40380000`, Espressif's custom UART peripheral at `0x60000000`.
- The firmware (`embedded/esp32c3/firmware/firmware.c`) talks to that UART directly, polling the TX FIFO count in `UART_STATUS_REG` bits 16..23 — completely different register layout from the 16550 used in the generic virt-machine target.

The kernel header, the Go-side reference, the `TinyModel` format, and the harness pattern are all **shared** between targets. Only the firmware wrapper differs:

```
embedded/
  gm1_avr.h                          # one kernel
  reference.go, model.go             # one Go reference
  iris/                              # one trained model
    data.go, train.go
  firmware/    (ATmega2560)          # AVR wrapper + simavr runner
  riscv/       (generic RISC-V virt) # rv32im wrapper + upstream QEMU
  esp32c3/     (real ESP32-C3)       # rv32imc wrapper + Espressif QEMU
```

Results

```
$ go test -v ./embedded/esp32c3/
=== RUN   TestESP32C3FirmwareMatchesReference
=== RUN   TestESP32C3FirmwareMatchesReference/setosa
=== RUN   TestESP32C3FirmwareMatchesReference/versicolor
=== RUN   TestESP32C3FirmwareMatchesReference/virginica
--- PASS: TestESP32C3FirmwareMatchesReference (0.62s)
PASS

$ go test -v ./embedded/riscv/
=== RUN   TestRISCVFirmwareMatchesReference
... PASS (1.0s, 3 subtests)

$ go test -v -run Accuracy ./embedded/riscv/
--- PASS: TestRISCVFirmwareAccuracyOnFullDataset (26.6s)
    qemu_harness_test.go:279: firmware Iris accuracy: 89/90 = 0.989

$ go test -v ./embedded/iris/
--- PASS: TestTrainFloatReachesReasonableAccuracy
    train_test.go:18: float model: train=1.000 test=0.944
--- PASS: TestQuantizedModelMatchesFloatWithinTolerance
    train_test.go:53: agreement float-vs-int8: 90/90 (100.0%);
                      float acc 0.989  int8 acc 0.989

$ go test -v ./embedded/
--- PASS: TestAVRInferenceMatchesReference (1.16s, 4 subtests)
```

What each test proves:

- **`TestAVRInferenceMatchesReference`** — the int8 kernel's arithmetic (including PROGMEM reads) is identical to the Go reference on a simulated 8-bit AVR with hand-picked deterministic weights.
- **`TestTrainFloatReachesReasonableAccuracy`** — the float SGD trainer produces a usable Iris classifier from a fixed seed.
- **`TestQuantizedModelMatchesFloatWithinTolerance`** — post-training quantization preserves every classification decision (no accuracy drop from float to int8 on this model).
- **`TestRISCVFirmwareMatchesReference`** — the same kernel runs on a bare-metal 32-bit RISC-V binary loaded into QEMU, classifying real Iris samples with output that matches the Go reference exactly.
- **`TestRISCVFirmwareAccuracyOnFullDataset`** — running every one of the 90 Iris samples through the firmware (one QEMU boot per sample) yields 89 correct classifications.
- **`TestESP32C3FirmwareMatchesReference`** — same as above, but on Espressif's QEMU fork with `-machine esp32c3` (real SoC peripherals, different UART driver, different memory map).

Footprint

Iris firmware on the ESP32-C3 build (`riscv64-unknown-elf-objdump -h`):

Section	Size	Notes
`.text`	~900 B	kernel + UART driver + `main`
`.rodata`	~220 B	weights (32 + 8 + 24 + 3 = 67 B) + format strings
`.bss`	~50 B	two int8 ping-pong buffers + int32 logits

Total under 2 KB on a chip with 400 KB SRAM and 4 MB flash. Headroom for ~2000× larger models without architectural changes.

Discovery highlights

A few things were not obvious going in:

- **simavr's `-machine` CLI does not install a console hook** even when the firmware uses `AVR_MCU_SIMAVR_CONSOLE`. The Debian 1.6 `simavr` binary does not call `avr_set_console_register`. We had to write a ~60-line libsimavr-based runner that hooks `UART_IRQ_OUTPUT` directly. This was discovered after three different "console register" attempts silently produced no output.
- **GCC emits `out` for low-I/O addresses on AVR**, bypassing simavr's store watch. Even when `avr_set_console_register` is wired up correctly, writing to GPIOR0 produced no output because `out 0x1e` doesn't trigger the SRAM watch. The fix was to drop the console-register approach entirely and use a real UART0 with a `UART_IRQ_OUTPUT` notify hook.
- **Espressif QEMU requires `-kernel`**, not `-bios`, for ELF firmware. Loading our binary with `-bios firmware.elf` triggered an internal assertion. The correct invocation is `qemu-system-riscv32 -machine esp32c3 -nographic -kernel firmware.elf` — and `-nographic`, not `-serial stdio`, otherwise QEMU refuses to multiplex the monitor and serial port.

- **ESP32-C3 has no SiFive test device** for clean QEMU exit. The RISC-V virt firmware writes `0x5555` to `0x100000` and QEMU exits immediately. On ESP32-C3 we instead make the firmware spin and have the Go harness kill QEMU as soon as it sees the `CLASS` line. Without that fix each ESP32-C3 subtest waited the full 10-second timeout; with it each subtest takes ~200 ms.

What is still missing

1. **Gorch → gm1 exporter.** The Iris pipeline trains in pure Go without touching any gorch code. Once gorch's training path is plumbed through, the same TinyModel emitter will accept a trained `*nn.Linear` stack. That work lives behind `//go:build darwin` since gorch is darwin-gated today.
2. **Larger models.** MNIST 784→64→10 (~51 KB weights, ~1.5 KB activations, ~20 ms per inference at 16 MHz on AVR) fits trivially on ESP32-C3. Need to add im2col-on-the-fly to support Conv2d.
3. **Real ESP32 (Xtensa) target.** Espressif QEMU also supports `-machine esp32` (the original LX6 dual-core). Same kernel, same harness pattern, but needs a different toolchain (`xtensa-esp32-elf-gcc`, ~50 MB).
4. **Accuracy regression alerts.** Currently each harness asserts a minimum accuracy threshold; a CI dashboard tracking per-target accuracy over time would catch quantization-related regressions that don't trip the floor.

File listing

```
embedded/
  README.md                   AVR target documentation
  gm1_avr.h                   shared int8 kernel
  reference.go, model.go      Go reference + TinyModel emitter
  avr_harness_test.go         AVR end-to-end test
  avr_runner/                 libsimavr-based AVR runner
    Makefile, runner.c
  firmware/                   ATmega2560 fixture firmware
    Makefile, firmware.c
  iris/
    data.go                   90 Iris samples (public domain)
    train.go                  SGD training + post-training quantization
    train_test.go             accuracy + agreement tests
  riscv/                      generic RISC-V virt machine (upstream QEMU)
    README.md, .gitignore
    qemu_harness_test.go
    firmware/
      Makefile, link.ld, start.S, firmware.c
  esp32c3/                    real ESP32-C3 (Espressif QEMU)
    .gitignore
    esp32c3_harness_test.go
    firmware/
      Makefile, link.ld, start.S, firmware.c
```

`gorch-microcontroller-report.pdf` `(this document, prior version)`

Branch: `claude/gorch-microcontroller-inference-Uw9vI` Commits: `eef0f1d` (AVR), `a160a23` (RISC-V virt + Iris), `e137012` (real ESP32-C3 via Espressif QEMU).